

Plan: HSIKAN Parameter Derivation

(collapsing ~ 25 env vars into a small set of fundamental knobs)

HyMeKo / HSiKAN

2026-05-12

1 Scope and goal

HSIKAN has accumulated approximately 25 environment variables plus 5 CLI arguments controlling cycle enumeration, vertex selection, attention, gates, kernels, training, and protocol — many of them coupled (e.g. `HSIKAN_TOPK_M_V_MAX` defaults to `HSIKAN_TOPK_K`; the various per-arity `MAX_K` caps are typically set together; `HSIKAN_CYCLE_BATCH` is a function of available memory). This makes experiments hard to reproduce, hard to compare across runs, and easy to misconfigure (today's morning OOM is a direct consequence of a single env-var addition that bypassed an implicit memory contract — see `project_rust_topk_memory_followup_2026_05_10`).

Concrete goal. Introduce an `HSIKANConfig` dataclass that exposes a small set of fundamental knobs (5-10) and *derives* the rest via documented rules. Each derived parameter has a `@cached_property` with the rule encoded in code. Env-vars remain as escape-hatch overrides so existing scripts keep working.

2 Affected files

- `signedkan_wip/src/config.py` — NEW. The `HSIKANConfig` dataclass + derivation rules + dataset-profile defaults.
- `signedkan_wip/src/run_final_cell.py` — replace the ~ 15 `os.environ.get(...)` calls at the top of `cell_signed_graph` with one `HSIKANConfig.from_env(g, args)`. Propagate the config object downstream; each callee that currently reads env-vars switches to `config.X`.
- `signedkan_wip/src/n_tuples.py` — the env-var dispatch in `_enumerate_cycles_fast` reads from a `HSIKANConfig` if provided, else falls back to env-vars. Backwards-compatible.
- `signedkan_wip/tests/test_config.py` — NEW. Tests for each derivation rule, dataset-profile defaults, env-var override priority, and round-trip serialisation.
- `signedkan_wip/src/mixed_arity_signedkan.py` — if any env-var reads exist here, switch to config dotted access.

3 CORE.YAML items touched

None. All affected files are in `signedkan_wip` which is not under `CORE.YAML`'s `crates/files/globs`.

4 The fundamental knobs (user-facing)

```
@dataclass
blue!60!blackclass HSIKANConfig:
    # ---- ALGORITHMIC (genuinely independent) ----
```

```

dataset: blue!60!blackstr                                # epinions /
    slashdot / ...
seed: blue!60!blackint = 0
n_epochs: blue!60!blackint = 80
hidden: blue!60!blackint = 4
mixed_tuples: blue!60!blacktuple = (red!50!black"red!50!blackc2red!50!black"
    ,red!50!black"red!50!blackc3red!50!black",red!50!black"red!50!blackc4
    red!50!black",red!50!black"red!50!blackc5red!50!black",red!50!black"
    red!50!blackw2red!50!black",red!50!black"red!50!blackw3red!50!black")
pruner: blue!60!blackstr = red!50!black"red!50!blackbalancerred!50!black"
scorer: blue!60!blackstr = red!50!black"red!50!blackfraction_negative
    red!50!black"
attention_kind: blue!60!blackstr = red!50!black"red!50!blackquaternion
    red!50!black"
highway_kind: blue!60!blackstr = red!50!black"red!50!blackedge_cr
    red!50!black"

# ---- BUDGET (one knob drives ALL caps) ----
memory_budget_gb: blue!60!blackfloat = 8.0
target_total_cycles: blue!60!blackint = 600_000

# ---- STRATEGY (one knob drives selection params) ----
selection_aggressiveness: blue!60!blackfloat = 0.5      # 0=legacy, 1=
    tiered+MSG

# ---- QUALITY (independent) ----
entropy_lambda: blue!60!blackfloat = 0.0
sparse_attn_k: blue!60!blackint | None = None
strict_protocol: blue!60!blackbool = False

# ---- BACKEND (independent toggles) ----
triton_kernel: blue!60!blackbool = True
triton_backward: blue!60!blackbool = True
direct_messaging: blue!60!blackbool = False
per_edge_gate: blue!60!blackbool = False

```

5 Derivation rules

From target_total_cycles → per-arity caps

FPN-style decay across cycle lengths — shorter cycles are abundant but local; longer cycles are rare but globally informative. Weights sum to 1.0:

$$\text{cap}_k = \text{target_total} \times w_k, \quad w \in \{3:0.35, 4:0.25, 5:0.18, 6:0.12, 7:0.10\}$$

From selection_aggressiveness → topk_mode + m_v_c + vertex_filter

agg.range	topk_mode	vertex_filter
[0.0, 0.2)	per_vertex	none
[0.2, 0.7)	per_vertex_adaptive	degree>=2
[0.7, 1.0]	per_vertex_tiered	compose:degree>=2, triangle

$$m_{v-c} = 1.0 + (0.5 - \text{aggressiveness})$$

From memory_budget_gb + graph stats → cycle_batch

At load time we know $|V|$, $|E|$, and `hidden`. We compute the expected per-chunk activation memory b_{chunk} and set:

$$\text{cycle_batch} = \max(500, \lfloor 0.3 \cdot \text{memory_budget_gb} \cdot 10^9 / b_{chunk} \rfloor)$$

From dataset → default profile

```
DATASET_PROFILES = {
red!50!black      red!50!black"red!50!blackbitcoin_alphared!50!black":
    HSIKANConfig(
        memory_budget_gb=1.0,
        target_total_cycles=50_000,
        selection_aggressiveness=0.3), # small graph, cheap to be
        liberal
red!50!black      red!50!black"red!50!blackbitcoin_otcred!50!black": HSIKANConfig
    (
        memory_budget_gb=1.0,
        target_total_cycles=50_000,
        selection_aggressiveness=0.3),
red!50!black      red!50!black"red!50!blackslashdotred!50!black": HSIKANConfig(
        memory_budget_gb=4.0,
        target_total_cycles=400_000,
        selection_aggressiveness=0.5),
red!50!black      red!50!black"red!50!blackepinionsred!50!black": HSIKANConfig(
        memory_budget_gb=8.0,
        target_total_cycles=400_000,
        selection_aggressiveness=0.6), # large graph, prefer
        concentration
}
```

6 Env-var override priority

1. If `HSIKAN_<FIELD>` is set in the environment, it wins.
2. Else if the field is in the dataset profile, use that.
3. Else use the dataclass default.

`HSIKANConfig.from_env(g, args)` reads the environment, layers the dataset profile, and produces a fully resolved config. The resolution is logged so the experiment is reproducible.

7 Interface changes

- `HSIKANConfig` is the only new public API.
- Existing scripts (overnight runners, 5-seed orchestrators) work unchanged: env-vars still parsed, just funneled through the dataclass.
- Reports include `config.to_json()` in provenance instead of the env-var dump.

8 Test strategy (CLAUDE.md §3)

Unit (tests/test_config.py)

- Each derivation rule has a dedicated test asserting the formula on hand-coded inputs.
- Env-var override priority: setting `HSIKAN_HIDDEN=8` beats the profile's `hidden=4`.

- Dataset-profile defaults: `HSIKANConfig.from_dataset("epinions")` has the documented fields.
- Round-trip: `HSIKANConfig.from_json(c.to_json()) == c`.
- `selection_aggressiveness=0.0` bit-equals the pre-derivation legacy env-var soup.

Integration

- Smoke run on a 200-vertex SBM fixture using `HSIKANConfig.from_dataset("sbm_n200_k4_s0")` produces the same AUC as the equivalent env-var config (within numerical noise).
- Run with `selection_aggressiveness=0.0` matches today's fixed-m=128 baseline numbers bit-for-bit on a deterministic seed.

Regression

- All existing 5-seed orchestrators in `signedkan_wip/experiments/*.sh` pass under the dataclass path (CI: each script runs at minimum its 1-seed=0 smoke).

9 Performance budget

This refactor is plumbing — no perf change. Test gate: `config.from_env(g, args)` resolution must complete in < 100 ms on Epinions (*g* already loaded).

10 Risk anticipation (CLAUDE.md §2)

Performance-contract preservation

The recently-landed `n_tuples._subsample_arr_to_tuples` (2026-05-10) pushes the `max_cycles` cap into the numpy-array subsample step before Python-tuple materialisation. The dataclass must preserve this: `config.cap_k[k]` is passed in unchanged to the existing call sites. Test: the post-refactor `construct_k` call site has the same `max_cycles` argument as the pre-refactor version.

The Rust-side memory followup (`project_rust_topk_memory_followup_2026_05_10`) is independent of this refactor.

Aspirational comments to honour

`n_tuples.py:367-369`: “Push the cap into the enumerator so we never materialise more than `max_cycles` Python tuples in memory.” This refactor does *not* address that aspirational comment — it's a separate Rust-side change. The config refactor sits cleanly above that work.

Worst-case input

Epinions production-config (`c2,c3,c4,c5,w2,w3,h=4,80` epochs, balance pruner, edge_cr highway, quaternion attention, triton kernel on, seed 0–4). This config's env-var dump is the canonical mapping target: `HSIKANConfig.from_dataset("epinions")` must yield exactly the same set of resolved parameters as today's `run_adaptive_mv_5seed_2026_05_10.sh`.

What could go wrong

- **Hidden env-var reads**: there are 25+ env vars; missing one in the dataclass leaves a silent default. Mitigation: a `config.warn_on_unrecognised_env()` pass that lists every `HSIKAN_*` env var on the system and warns if any aren't consumed by the dataclass.

- **Bit-equivalence break:** a derivation rule produces a slightly different value than the env-var default (e.g., `cap_k[4]` is 100 000 today by CLI default but derivation puts it at 150 000). Mitigation: dedicated bit-equiv test at `selection_aggressiveness=0.0` pinning every resolved field to today's defaults.
- **Dataset-profile coupling:** `epinions` profile sets `memory_budget_gb=8.0` which on a smaller machine would OOM. Mitigation: profiles can be overridden by env-vars; document this in `config.py` module docstring.

11 Empty-plan-dir hygiene

This plan dir is fully populated. If abandoned, replace with a `HALTED.md` containing the abort reason.

12 Rollback path

The refactor is additive: `HSIKANConfig` is a new module. Existing env-var parsing in `run_final_cell.py` is preserved as the fallback path. Reverting the patch removes `config.py` and restores the original env-var-soup top of `cell_signed_graph`. Existing scripts continue to work in the rollback state.

13 Halt conditions

- Bit-equiv test fails at `selection_aggressiveness=0.0` — a derivation rule has the wrong default; halt and reconcile before merging.
- Any of today's queued scripts produces a different AUC under the dataclass path vs the env-var path — silent semantic regression; halt.
- Documented env-var-override priority isn't observed in a test — the layered resolution has a bug.

14 Implementation order (v1 / v2 / v3)

v1 (this plan)

1. `HSIKANConfig` dataclass + derivation rules.
2. `from_env_or_dataset` resolution + override priority.
3. All unit tests; bit-equiv test at `aggressiveness=0.0`.
4. `cell_signed_graph` top swapped to dataclass path.
5. Single-seed smoke on `Epinions` production config matching today's baseline AUC.

v2 (separate plan)

1. Expose `config` as a CLI flag (`--config path/to/json`) for full-experiment reproducibility.
2. Wire `HSIKANConfig` into the orchestrator scripts — generate scripts from config instead of duplicating env-var blocks.
3. Add a registry of named configs that produced published numbers (e.g., `slashdot_edge_cr_5seed_2026_05_0`) so each SOTA-producing experiment has a single-file reproduction artifact.

v3 (much later)

1. Auto-tuning: given `memory_budget_gb` + dataset, run a short calibration to find the `selection_aggressiveness` that maximises AUC under the budget. Output: a profile-derived config.